

React Native Messages Directory

1. What This Project Is About

So for Exercise 3 I built a messages directory app using React Native and Expo. The idea is pretty simple - you get a home screen with 6 different directory categories shown as colourful cards, and when you tap on any of them it takes you to a messages screen that shows all the stored messages for that category. I went with a modern dark navy header design instead of just copying the basic circle layout from the reference screenshot because I wanted it to look like an actual app someone would use.

The whole thing is built using proper React Native components, stack navigation for moving between screens, and a clean file structure with separate screen files and a data file rather than putting everything into one App.js.

2. Features of the App

- Home screen with 6 directory cards in a 2-column grid layout, each with its own unique colour and icon
- A working search bar at the top that filters the directory cards in real time as you type, with a clear button to reset it
- Each card shows the category name, icon, and the number of messages dynamically calculated from the data file
- Tapping any card navigates to the messages screen for that category using stack navigation
- Messages screen shows all messages as individual cards with a numbered colour-matched badge, the message text, and a timestamp
- Custom back button in the top left of the messages screen to return to the home screen
- Empty state screen that appears when you search for something that does not match any directory
- Status bar styled with light content to match the dark navy header
- 5 messages per category with varying timestamps like Today, Yesterday, Mon, and so on to make it look realistic
- Total message count shown dynamically in the header subtitle on the home screen

3. Project Structure

This is the complete folder layout of the project. I kept the screen files separate from the data and kept App.js as minimal as possible since all the real logic lives in the screen files.

```

MessagesDirectory/
├─ App.js                ← Entry point, sets up navigation container
├─ app.json              ← Expo configuration
├─ package.json          ← Project dependencies
├─ index.js              ← App entry point
|
├─ screens/
|   ├─ HomeScreen.js     ← Home screen with 6 directory cards and search
|   └─ MessagesScreen.js ← Messages list for selected category
|
├─ data/
|   └─ messages.js       ← All message data for all 6 categories
|
└─ assets/
    ├─ icon.png
    ├─ splash-icon.png
    └─ android-icon-background.png

```

App.js This is just the entry point that sets up the `NavigationContainer` and the `Stack.Navigator` with two routes. I set `headerShown` to `false` here because each screen handles its own custom header styling instead of using the default React Navigation header.

screens/HomeScreen.js This is where the main directory grid lives. It renders the 6 category cards using a `FlatList` with `numColumns` set to 2. It also handles the search filtering logic using `React useState` and includes the empty state for when nothing matches. When a card is tapped it passes the category name, colour, and icon through to the messages screen via route params.

screens/MessagesScreen.js This screen receives the category, colour, and icon from the home screen through navigation params. It looks up the right messages from the data file and renders them in a `FlatList` as individual styled cards. The header colour and icon match the category that was tapped.

data/messages.js This is a plain JavaScript object with 6 keys, one for each category. Each key holds an array of 5 message objects with an `id`, `text`, and `time` field. Keeping data separate from the UI made it easy to update messages without touching any screen code.

4. How to Configure and Run

What you need installed before starting:

- Node.js version 18 or higher
- npm version 9 or higher
- JDK 17 or higher
- Android Studio with an AVD emulator configured

- The ANDROID_HOME environment variable set to your Android SDK path

On Windows you need to set these environment variables manually. Go to System Environment Variables and add ANDROID_HOME pointing to your SDK folder, then add the platform-tools and emulator folders to your Path variable. In my case the SDK was at C:\Android\Sdk so I set it to that.

Steps to run:

Step 1 - Extract the ZIP or clone from GitHub into a folder on your computer

Step 2 - Open a terminal, navigate into the project folder and install all the dependencies:

```
cd MessagesDirectory
```

```
npm install
```

```
npx expo install react-native-screens react-native-safe-area-context react-native-gesture-handler
```

```
npm install @react-navigation/native @react-navigation/stack
```

```
npx expo install @expo/vector-icons
```

Step 3 - Open Android Studio, go to Virtual Device Manager, and press the Play button on your emulator. Wait until the Android home screen fully loads before doing the next step.

Step 4 - Start the app:

```
npx expo start --android
```

Expo will start Metro Bundler, automatically install Expo Go on the emulator, and launch the app. The first time it runs it takes a minute or two to bundle everything. After that it is much faster.

Step 5 - Use the app by tapping any of the 6 directory cards to open the messages screen. Use the back arrow in the top left to return. You can also type in the search bar to filter the directories in real time.

5. Technologies Used

- React Native - the main framework used to build the app
- Expo SDK 56 - used to manage the project, run on the emulator, and handle native modules
- JavaScript with JSX - the language everything is written in
- React Navigation - used for the stack navigator and screen transitions
- Ionicons from @expo/vector-icons - used for the icons on each category card and the back button
- Android Studio - used for the emulator and testing
- Node.js and npm - used for package management
- Git and GitHub - used for version control and submission

6. Challenges I Ran Into

The first problem was the `ANDROID_HOME` environment variable not being set correctly. When I first ran `npx expo start --android` it kept printing an error saying it could not find the Android SDK at the default path. I had to manually go into Windows System Environment Variables and add `ANDROID_HOME` pointing to `C:\Android\Sdk` and also add the `platform-tools` and `emulator` subfolders to the `Path` variable. Once I opened a fresh terminal after saving those changes, `adb` started working and the app launched on the emulator.

Another issue was the content overlapping the status bar at the top of the screen. The header text was sitting right behind the carrier signal and battery icons which looked wrong. I fixed this by removing `SafeAreaView` and instead adding a `paddingTop` of 52 directly to the header style so it always clears the status bar properly.

The emulator kept showing a System UI not responding dialog every time I tried to interact with it. This is just the emulator being slow due to RAM limitations, not anything wrong with the app itself. Clicking Wait each time fixed it and the app ran normally. I could have also configured the emulator to use more RAM in the AVD settings to reduce how often this happens.

I also ran into a situation where the `@expo/vector-icons` package was not resolving even though Expo normally includes it. The error said it could not find the module in the `node_modules` directory. Running `npx expo install @expo/vector-icons` separately fixed it and the icons loaded correctly after that.

7. Responsive Design

The app handles different screen sizes using `FlatList` with `numColumns` set to 2 for the home screen grid. The card width is set to 48 percent so it always fills the available space with a small gap between cards regardless of screen width. The messages screen uses a standard `FlatList` that scrolls naturally if there are more messages than fit on the screen. `SafeAreaView` was replaced with a plain `View` plus manual `paddingTop` on the header so the layout works consistently across different Android devices with different status bar heights.

8. GitHub Repository

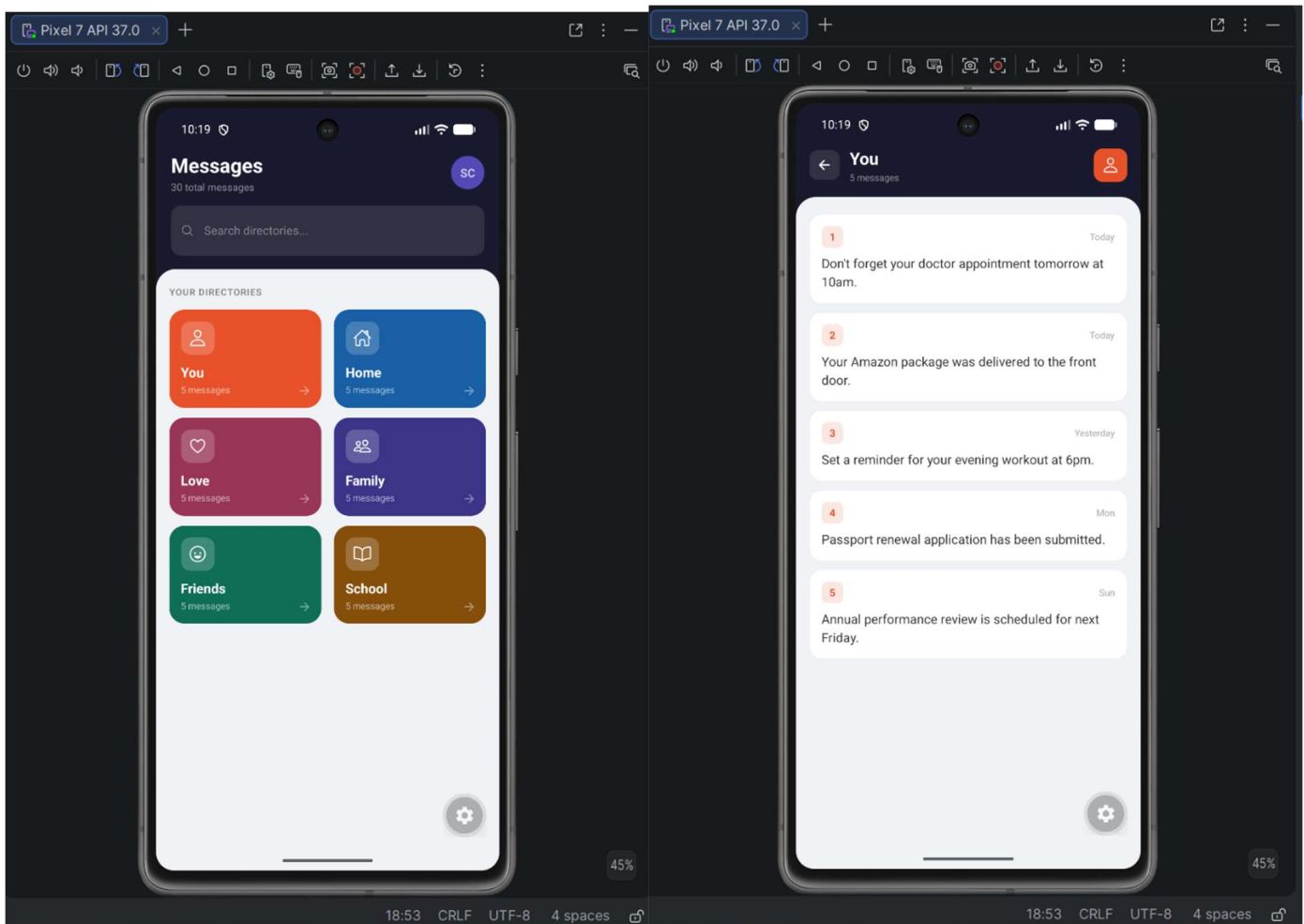
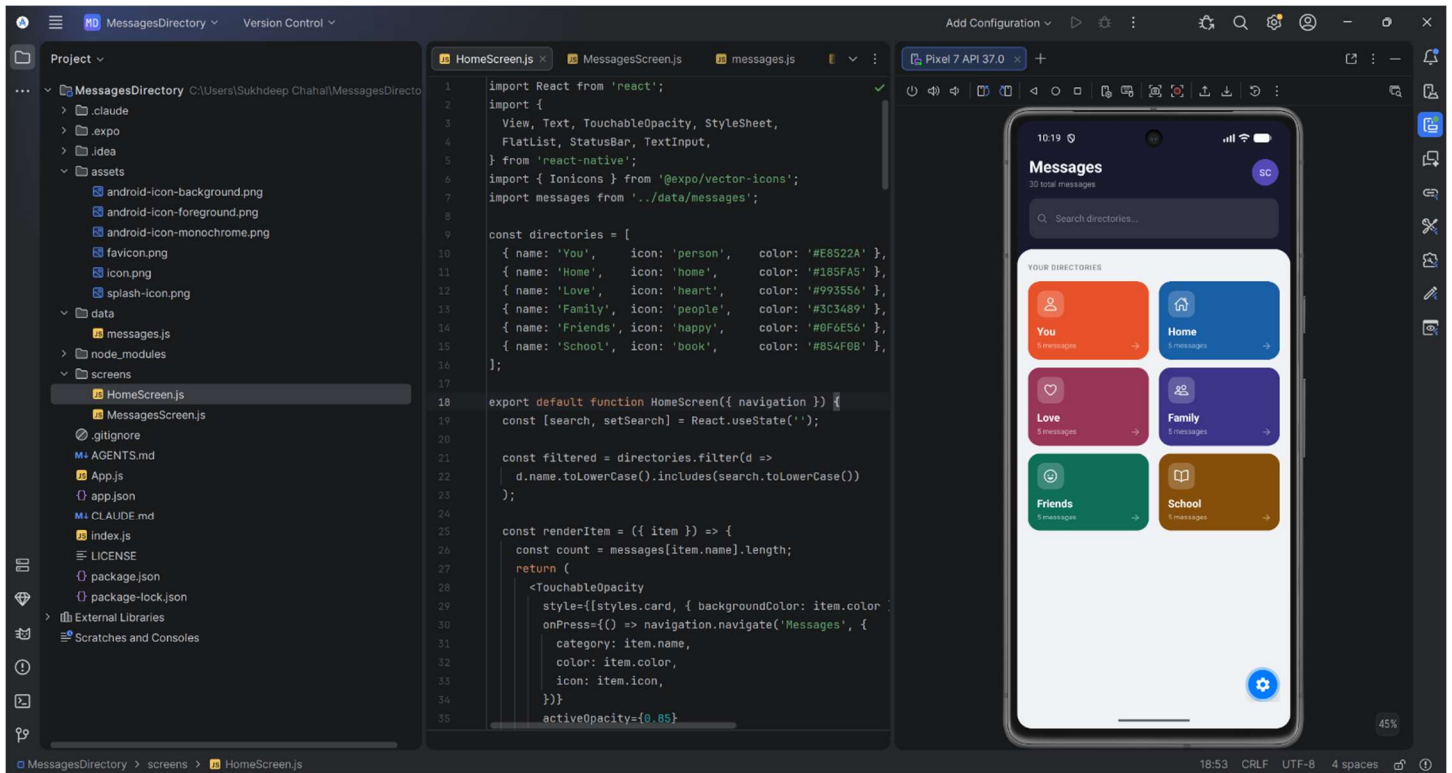
Full project including all JS source files, assets, and this README is available at:

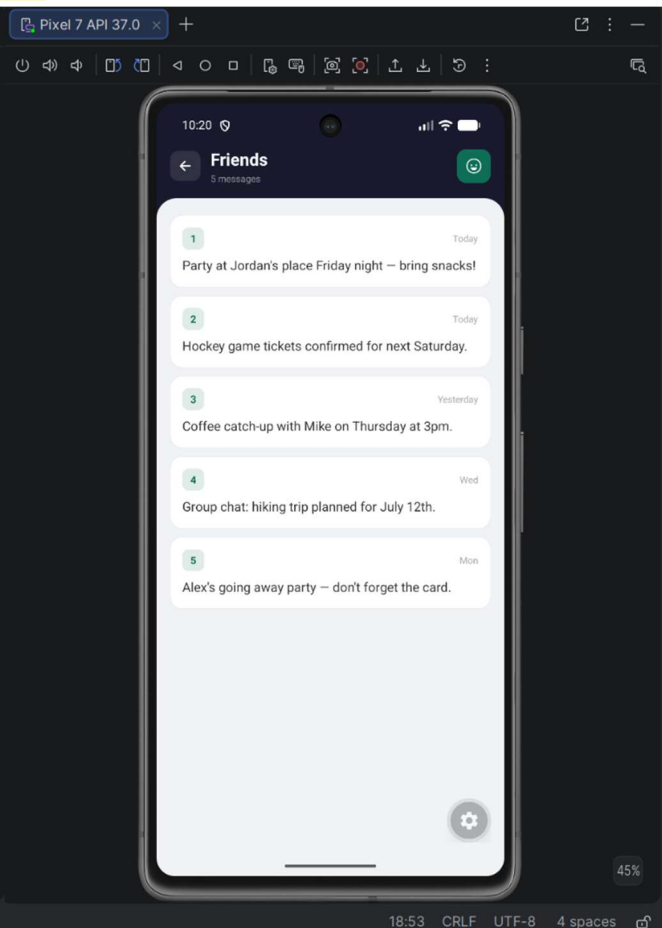
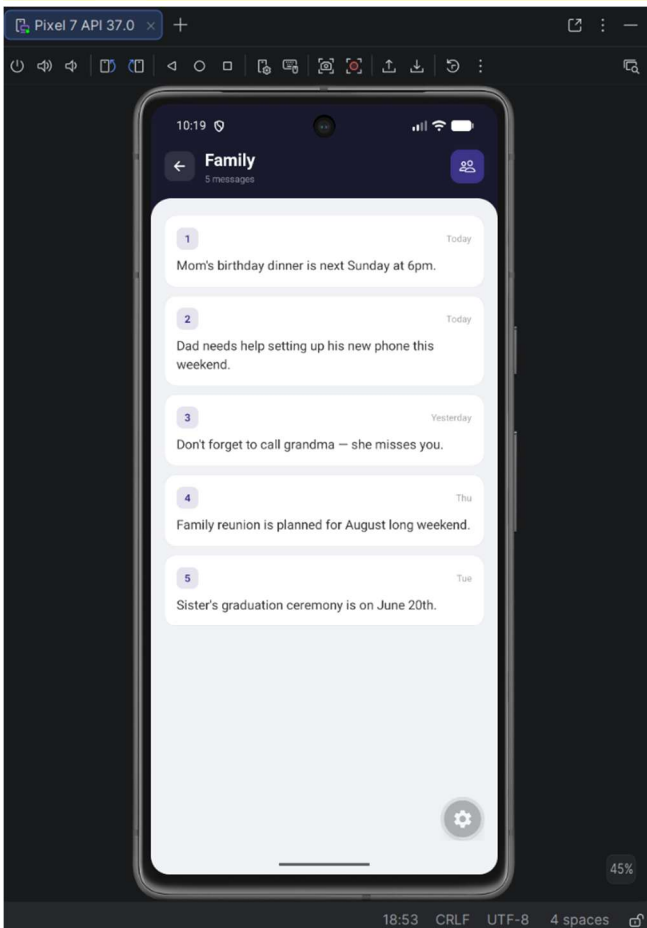
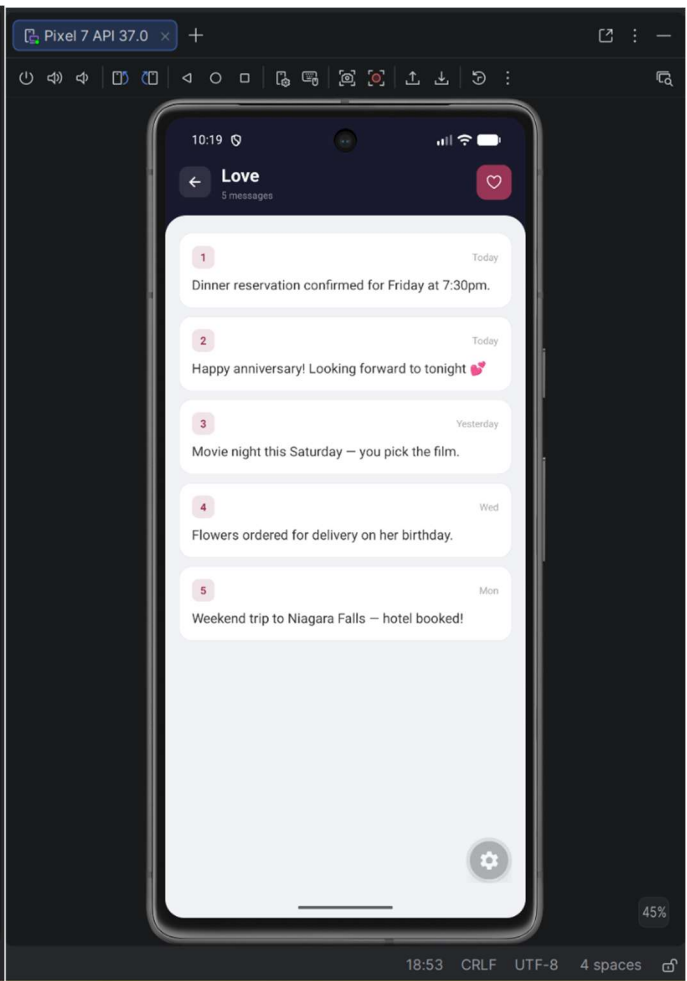
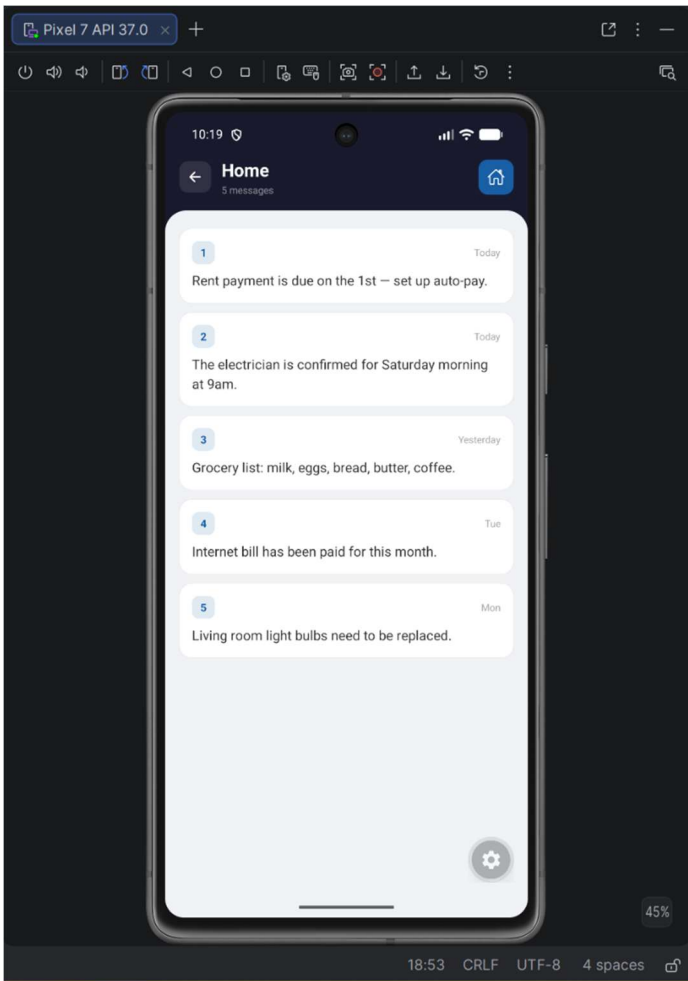
<https://github.com/Sukhdeep20/MessagesDirectory>

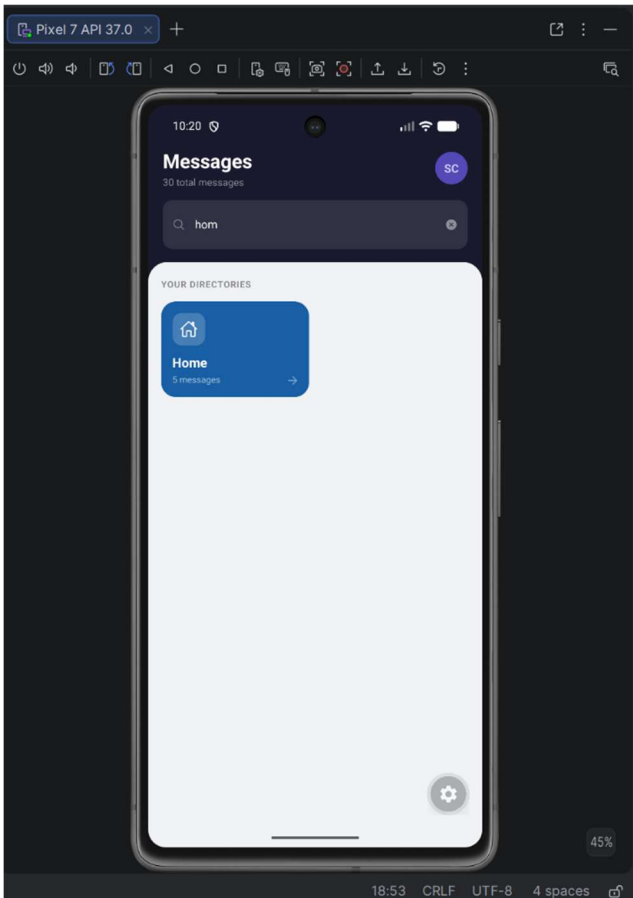
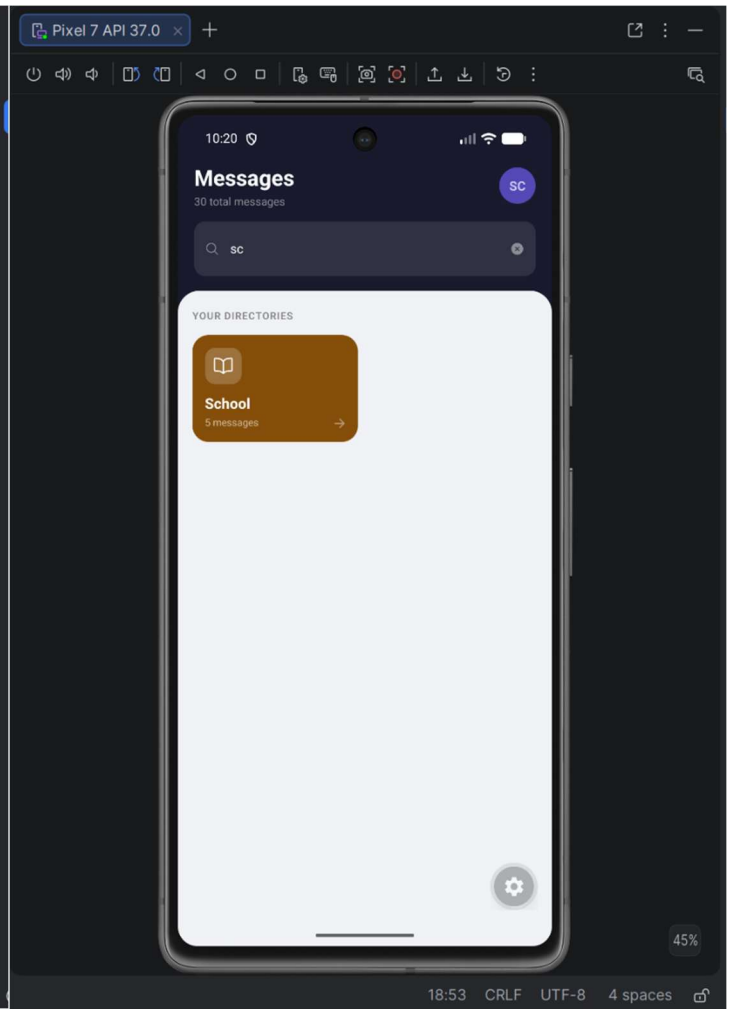
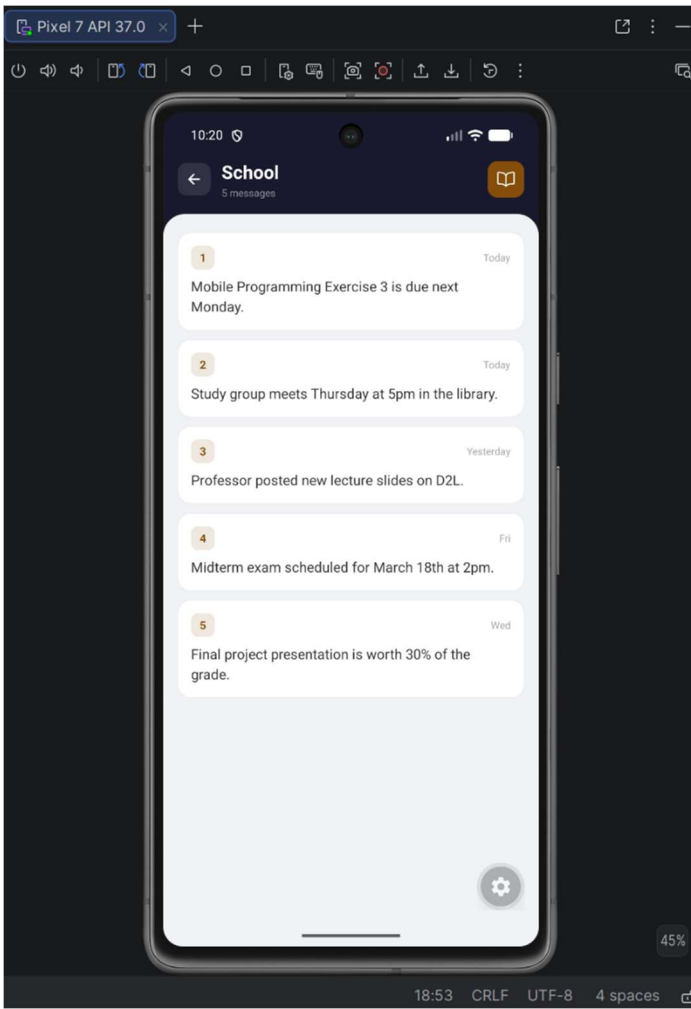
9. What I Would Add With More Time

- The ability to add new messages to a category instead of just reading static ones
- Swipe to delete individual messages
- Different colour themes the user can switch between
- Animations when navigating between the home and messages screens
- Local storage using `AsyncStorage` so messages persist between app sessions
- A badge on each category card showing unread message count

10. Screenshots







Submitted By:

Sukhdeep Singh

1307039

Mobile Programming - CS5450

Lakehead University - Spring 2026